

# PRÁCTICA 11

## Procesamiento de imágenes

Parte 3

### ■ Descripción de la práctica

El objetivo de esta práctica es realizar una aplicación que amplíe la realizada anteriormente, introduciendo nuevas funcionalidades relativas al procesamiento imágenes. Concretamente, deberá incluir las siguientes nuevas funcionalidades:

- Operadores de transformación (variaciones de brillo y contraste)
- Rotación y escalado de imágenes

El aspecto visual de la aplicación será el mostrado en la Figura 1. El menú incorporará en su opción “Imagen”, además de lo incluido en la practica 10, los ítems “AffineTransformOp”, “LookupOp”. En la parte inferior, además de los ya incluido en la práctica 10, se incorporarán tres botones asociados a tres tipos de contraste (normal, iluminado y oscurecido), tres deslizadores y un botón para un dos nuevos operadores, un botón para rotación de 180° y dos botones para el escalado (incrementar y reducir).

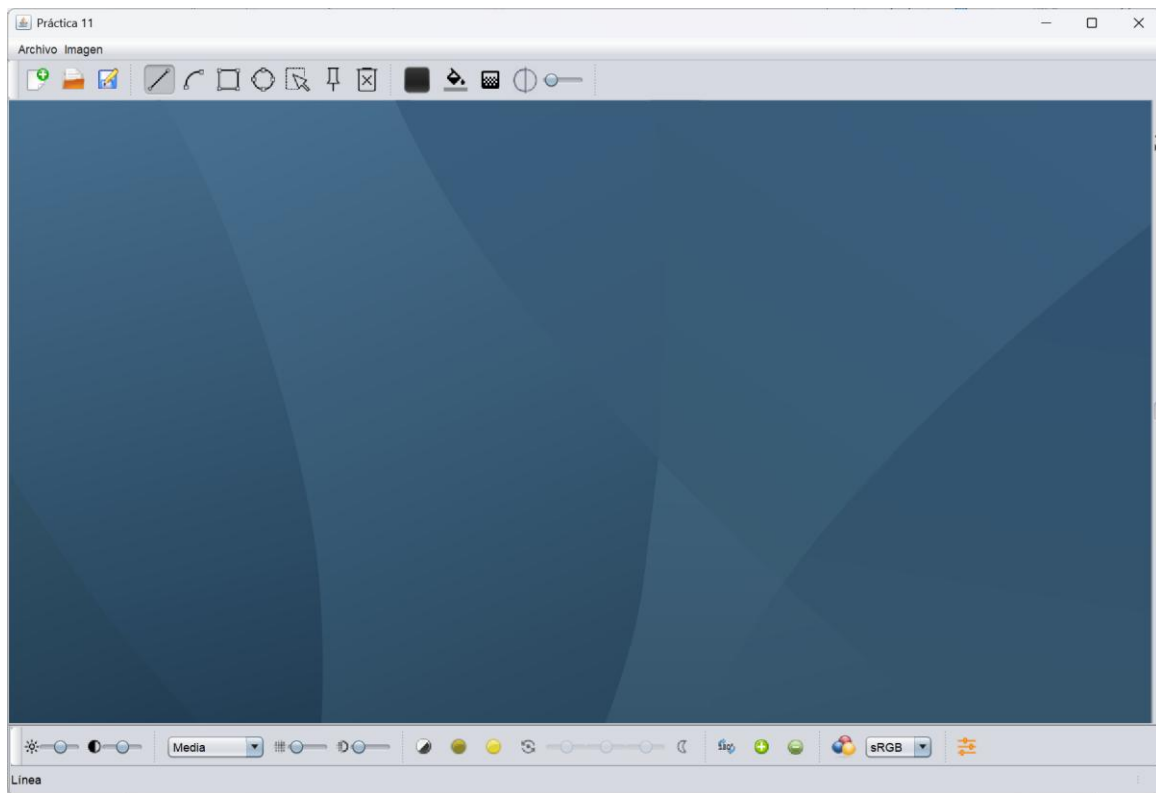


Figura 1: Aspecto de la aplicación

### ■ Pruebas iniciales

En una primera parte, probaremos los operadores “*AffineTransformOp*” y “*LookupOp*” usando parámetros fijos (i.e., sin interacción del usuario para definir sus valores). Para ello, incluiremos las correspondientes opciones en el menú “Imagen” cuya selección implicará la aplicación de la correspondiente operación en la imagen seleccionada.

En estos casos, se probará el código mostrado en las transparencias de teoría (que incluiremos en el manejador del evento “acción” asociado al menú), teniendo en cuenta que dicha operación se aplicará sobre la imagen mostrada en la ventana activa. Al igual que vimos en la práctica 10, esto supondrá que, al código visto en teoría, habrá que añadirle las sentencias para el acceso y actualización de la imagen; por ejemplo, para el caso de la operación “AffineTransformOp” y el escalado:

```
private void menuAffineTransformOpActionPerformed(ActionEvent evt) {
    VentanaInterna vi = (VentanaInterna) (escritorio.getSelectedFrame());
    if (vi != null) {
        BufferedImage img = vi.getLienzo2D().getImage();
        if (img != null) {
            try {
                AffineTransform at = AffineTransform.getScaleInstance(1.5, 1.5);
                AffineTransformOp atop = new AffineTransformOp(at, null);
                BufferedImage imgdest = atop.filter(img, null);
                vi.getLienzo2D().setImage(imgdest);
                vi.getLienzo2D().repaint();
            } catch (IllegalArgumentException e) {
                System.err.println(e.getLocalizedMessage());
            }
        }
    }
}
```

Obsérvese que en el código anterior se ha optado por pasarle *null* como segundo parámetro al método *filter*, con lo cual se crea una nueva imagen salida; esto implica que sea necesario actualizar la imagen del lienzo de forma explícita. Por otro lado, en la creación del operador no lo indicamos ningún método de interpolación específico (al usar *null*, emplea el método de “vecino más cercano” por defecto); si queremos mejorar el resultado usando otra interpolación, podría indicarse mediante las variables estáticas que ofrece la clase *AffineTransformOp* (por ejemplo, *AffineTransformOp.TYPE\_BILINEAR* para interpolación bilineal<sup>1</sup>).

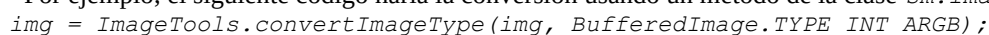
Para el caso de la operación “LookupOp”, el código será similar al anterior, pero cambiando el operador; por ejemplo, para aplicar una operación de negativo (véanse transparencias) el código sería:

```
byte funcionT[] = new byte[256];
for (int x=0; x<256; x++)
    funcionT[x] = (byte) (255-x); // Negativo
LookupTable tabla = new ByteLookupTable(0, funcionT);
LookupOp lop = new LookupOp(tabla, null);
```

En relación al operador *LookupOp*, hay que tener en cuenta que para imágenes tipo “BGR” (p.e., *TYPE\_INT\_BGR*), si no indicamos imagen de salida en la llamada a *filter* (i.e, dejamos a *null* el segundo parámetro), el operador genera una imagen de salida que intercambia<sup>2</sup> los canales B y R. Para abordar este problema, podríamos optar por convertir la imagen fuente a tipo *TYPE\_INT\_RGB* para asegurar compatibilidad<sup>3</sup> o pasar en la llamada a *filter* una imagen destino compatible con la fuente (como caso particular, se podría pasar la misma imagen fuente, que en ese caso sería modificada).

<sup>1</sup> En este caso, además, emplearía transparencia en la imagen resultado (esto es relevante, por ejemplo, en el caso de rotaciones)

<sup>2</sup> Bug no documentado

<sup>3</sup> Por ejemplo, el siguiente código haría la conversión usando un método de la clase *sm.image.ImageTool*:  


## ■ Variación del brillo y contraste

En este apartado modificaremos el contraste de la imagen aplicando el operador *LookupOp*. Para ello, incluiremos tres botones correspondientes a tres posibles situaciones:

- Contraste “normal”, para imágenes en las que la luminosidad esté equilibrada. En este caso, se usan funciones tipo S
- Contraste con iluminación, para imágenes oscuras. En este caso, se usan funciones tipo logaritmo (si es muy oscura), funciones raíz (con cuyo parámetro podemos determinar el grado de iluminación) o correcciones gamma (con gamma entre 0 y 1)
- Contraste con oscurecimiento, para imágenes sobre-iluminadas. En este caso, se usan funciones potencia (con cuyo parámetro podemos determinar el grado de oscurecimiento) o correcciones gamma (con gamma mayor que 1).

Cada una de las operaciones anteriores está asociada a una determinada función (véanse transparencias de teoría). En la clase *LookupTableProducer* del paquete *sm.image* se definen métodos estáticos para crear objetos *LookupTable* correspondientes a funciones estándar (negativo, función-S, potencia, raíz, corrección gamma, etc.); por ejemplo, para crear la función S pasándole los correspondientes parámetros usaríamos el siguiente código<sup>4</sup>:

```
| LookupTable lt = LookupTableProducer.sFuction(128.0,3.0);
```

Por otro lado, esta clase define un método *createLookupTable* que devuelve objetos *LookupTable* correspondientes a las funciones clásicas anteriores pero usando parámetros predefinidos. Por ejemplo, el siguiente código crearía un objeto *LookupTable* por defecto asociado a la función-S:

```
| LookupTable lt;  
| lt=LookupTableProducer.createLookupTable(LookupTableProducer.TYPE_SFUNCTION);
```

Para los ejemplos que se proponen en este ejercicio, bastaría usar los parámetros por defecto que ofrece el método *createLookupTable*. Por ejemplo, para el caso del contraste normal<sup>5</sup>:

```
| private void bContrasteActionPerformed(ActionEvent evt) {  
|     VentanaInterna vi = (VentanaInterna) (escritorio.getSelectedFrame());  
|     if (vi != null) {  
|         BufferedImage img = vi.getLienzo2D().getImage();  
|         if (img!=null){  
|             try{  
|                 int type = LookupTableProducer.TYPE_SFUNCTION;  
|                 LookupTable lt = LookupTableProducer.createLookupTable(type);  
|                 LookupOp lop = new LookupOp(lt, null);  
|                 lop.filter( img , img); // Imagen origen y destino iguales  
|                 vi.getLienzo2D().repaint();  
|             } catch(Exception e){  
|                 System.err.println(e.getLocalizedMessage());  
|             }  
|         }  
|     }  
| }
```

Nótese que en ejemplo anterior se usa como imagen destino la propia imagen fuente.

---

<sup>4</sup> Por si resulta útil de cara a implementar otras funciones propias, el código en el paquete *sm.image* correspondiente a la función S es el siguiente:

```
| public static LookupTable sFuction(double m, double e){  
|     double Max = (1.0/(1.0+Math.pow(m/255.0,e)));  
|     double K = 255.0/Max;  
|     byte lt[] = new byte[256];  
|     lt[0]=0;  
|     for (int l=1; l<256; l++){  
|         lt[l] = (byte) (K*(1.0/(1.0+Math.pow(m/(float)l,e))));  
|     }  
|     ByteLookupTable slt = new ByteLookupTable(0,lt);  
|     return slt;  
| }
```

<sup>5</sup> Si se quisiera modificar los parámetros del contraste, habría que llamar directamente a la función S (sin llamar al método *createLookupTable*) pasándole los parámetros correspondientes.

## ■ Rotación y escalado

En este apartado incluiremos la posibilidad de que el usuario pueda rotar y/o escalar imágenes usando valores fijos predeterminados. En primer lugar, incluiremos un botón para poder rotar la imagen 180°; para ello haremos uso del operador “*AffineTransformOp*”, teniendo en cuenta que la rotación tendrá que hacerse poniendo como eje de rotación el centro de la imagen<sup>6</sup>:

```
double r = Math.toRadians(180);
Point c = new Point(imgSource.getWidth()/2, imgSource.getHeight()/2);
AffineTransform at = AffineTransform.getRotateInstance(r,p.x,p.y);
AffineTransformOp atop;
atop = new AffineTransformOp(at,AffineTransformOp.TYPE_BILINEAR);
BufferedImage imgdest = atop.filter(imgSource, null);
```

Además, permitiremos escalar la imagen mediante dos botones: uno para aumentar el tamaño de la imagen y otro para reducirlo. De nuevo, usaremos un operador “*AffineTransformOp*”, en este caso creando un escalado donde fijaremos el factor de escala (por ejemplo, 1.25 para aumentar y 0.75 para reducir).

## ■ El reto final...

Por último, se proponen los siguientes retos que te permitirán confirmar si has entendido bien cómo se crea un operador basado en función:

1. **Transformación lineal con tres puntos de control.** Implementar un operador “*LookupOp*” que aplique la siguiente transformación  $T: [0,255] \rightarrow [0,255]$ , con parámetros  $a, b, c \in [0,255]$ :

$$T(x; a, b, c) = \begin{cases} \frac{(b-a) \cdot x}{128} + a & \text{si } x < 128 \\ \frac{(c-b)(x-128)}{127} + b & \text{si } x \geq 128 \end{cases}$$

La función anterior corresponde a un spline lineal que pasa por los puntos (0,a), (128,b) y (255,c). Se aconseja inicializar<sup>7</sup> a=0, b=128 y c=255. Nótese que se fijan las coordenadas X y lo que varía son las coordenadas Y. ¿Sabrías explicar qué efecto tiene este operador sobre la imagen y cómo afectan los parámetros “a”, “b” y “c”?<sup>8</sup>. Como casos particulares, trata de justificar qué efecto se obtendría para los siguientes valores: (1) a=b=c=128; (2) a=0, b=128, c=255; (3) a=255, b=128, c=0; (4) a=255, b=128, c=255; (5) a=0, b=128, c=0.

En la aplicación incluiremos un botón de dos posiciones para activar/desactivar este operador (véase Figura 1)<sup>9</sup>; junto al botón, se incluirán tres deslizadores con los que indicar los valores de los parámetros a, b y c de la ecuación. Al activar la operación, el usuario podrá ir variando los valores de los parámetros e ir viendo el efecto sobre la imagen que haya en el lienzo al comienzo de la operación; el resultado se dará por definitivo cuando se pulse de nuevo el botón y se desactive el operador.

Para entender mejor la función aplicada y su efecto, lanza un diálogo que muestre la función. Para ello, utiliza la clase *DialogoFuncionABC* del paquete *sm.iu*. Define y crea el diálogo en la ventana principal:

```
| DialogoFuncionABC dlgFuncion = new DialogoFuncionABC(this);
```

<sup>6</sup> Tras aplicar el *AffineTransformOp*, la imagen resultado tendrá activo el canal alfa.

<sup>7</sup> Suele ser útil utilizar herramientas de visualización de funciones (como, por ejemplo, [Mahtway](#)) para así entender gráficamente el comportamiento de la función y la influencia de los parámetros.

<sup>8</sup> Para dar respuesta a estas preguntas, se aconseja dibujar la gráfica para diferentes valores de los parámetros.

<sup>9</sup> Nótese que, en este caso, el inicio y fin de operación estarán asociados al evento de *actionPerformed* del botón, distinguiéndose el inicio o fin en función de que esté o no seleccionado. Es un enfoque distinto al que usamos para el brillo en la práctica 9, donde se asociada el inicio/fin a ganar o perder el foco, si bien la acción a realizar es la misma: crear una copia de la imagen al inicio y liberarla al final.

y actívalo cuando se inicie la operación

```
| dlgFuncion.setABC(a, b, c); //a, b y c los valores del slider  
| dlgFuncion.setVisible(true);
```

Cuando cambie el valor de los parámetros (gestionado en el *StateChanged* de los deslizadores) cambia el valor de A, B o C mediante los métodos *set* correspondientes. Por ejemplo, para cambiar el valor de A:

```
| dlgFuncion.setA(a); //a el valor del slider
```

Por último, recuerda desactivar el diálogo al acabar la operación:

```
| dlgFuncion.setVisible(false);
```

2. **Invertir zonas oscuras.** ¿Qué valores habría que darles a los parámetros a, b y c del ejercicio anterior para conseguir el efecto “*invertir el color de las zonas oscuras manteniendo el de las zonas claras*”? Incluir en la ventana principal un botón que, al pulsarlo, aplique este efecto a la imagen que se esté mostrando en ese momento en el lienzo. Incluir un comentario en la cabecera del método que justifique por qué la solución responde a la operación que se solicita

## ■ Para trabajar en casa...

Una vez realizada la práctica, algunas mejoras a considerar podrían ser las siguientes:

- Incluir la operación “negativo”.
- Incluir una opción “duplicar” que cree una nueva ventana interna con una copia de la imagen que había en la ventana activa.
- Definir nuevas operaciones “lookup” propias.